

A Coq Verification Blueprint for Option-Market Matching and Atomic Complex Orders

Ali Nehrani
Revised draft

July 5, 2026

Abstract

Electronic option exchanges match contract series rather than undifferentiated assets and must also process multi-leg strategies such as vertical spreads, straddles, butterflies, collars, and covered calls. Mechanical correctness therefore requires both ordinary order-book invariants and package-level ratio preservation: unless an instruction explicitly permits legging, a matching engine must not turn a complex order into a broken strategy.

This paper presents a Coq verification blueprint and an initial Rocq prototype for option-market microstructure. The specification covers typed option contracts, single-series orders, continuous matching, uniform-price opening selection, and complex orders represented as signed leg bundles. The target properties are contract consistency, expiry and tick validity, no overfill, individual rationality, price-time fairness, maximum executable opening volume, and multi-leg consistency. Following the faithfulness-audit methodology used by Coelho’s Lean 4 mathematical-finance library, every prototype result is classified as *full*, *library wrapper*, *reduced core*, or *placeholder*. At the present revision, the build contains 17 audited declarations: 10 full or wrapper results, one reduced core, and six admitted proof obligations. In particular, candidate-price maximality and a local contract-preservation lemma are checked, while end-to-end no-overfill, full opening allocation, and the executable complex-order matcher remain incomplete. The contribution is therefore a precise architecture, an auditable mechanization baseline, and a central ratio-preservation invariant—not yet a fully verified exchange engine.

Keywords: option markets; double auctions; formal verification; Rocq; Coq; matching engines; complex orders; opening auction; market microstructure.

1 Introduction

Modern financial exchanges are algorithmic institutions. They receive orders, rank them, match compatible buyers and sellers, allocate quantity, and report trades. For ordinary double auctions, recent formal-verification work has already shown that useful market-microstructure properties can be specified and proved in a theorem prover. In particular, Natarajan, Sarswat, and Singh formalize multi-quantity double-sided auctions in Coq, prove fairness, individual rationality, uniform matching, maximum matching, and uniqueness-style results, and extract certified programs that can be used to detect violations in exchange outputs [1]. Garg et al. later extend this line of work with automated checkers for double auctions [2]. These papers provide the closest formal foundation for the present project.

Options markets require additional structure. A standard equity option contract is not merely an order for an abstract commodity. It is a series determined by an underlying security, strike, expiration, call-or-put type, exercise style, and contract multiplier. OCC product specifications state,

for example, that standard equity option contracts cover 100 shares, quote premiums in points, use strike intervals, and are American-style in the ordinary listed-equity setting [5]. Therefore, a bid for one option series is not compatible with an ask for a different strike, expiration, option type, or underlying.

The second layer of complexity is package execution. Options venues define and process complex orders involving concurrent execution of multiple series. Cboe’s rulebook defines a complex order as an order or quote involving concurrent execution of two or more different series in the same underlying security or index [6]. Cboe also distinguishes instructions such as all-or-none complex orders and “Complex Only” orders, and imposes execution-price protections for complex orders [6]. These are not corner cases. SEC market-structure materials report customer options executions by sale condition and identify multi-leg electronic and multi-leg auction executions as substantial categories of dollar volume [7]. Thus, a verified theory of option-market matching must handle both single-series matching and package-level invariants.

This paper proposes such a theory and begins its mechanization. The aim is not option pricing. We do not model stochastic dynamics, implied volatility, early exercise, delta hedging, or risk-neutral valuation. Instead, we isolate the microstructure problem:

When is an option matching engine mechanically correct?

The intended answer is the following proof-obligation package:

contract consistency + expiry validity + tick validity
+ no overfill + individual rationality + fairness
+ maximum uniform auction volume + complex-order leg consistency.

The main new property is complex-order leg consistency. In a stock auction, a trade is usually a single matched buyer–seller pair. In an options market, a trader may submit a vertical spread, butterfly, straddle, strangle, collar, or covered call as one strategy. The matching engine must not fill the long call while failing to fill the short call, unless the submitted instruction explicitly permits legging or partial package execution. Package-level atomicity and ratio preservation are therefore the option-market analogue of no overfill and individual rationality.

Contributions. The paper makes the following contributions.

- (C1) We specify an option-contract and order model that makes series identity, expiry, price ticks, and package instructions explicit.
- (C2) We implement an initial Rocq prototype for single-series matching, candidate-price selection, and signed complex-order legs.
- (C3) We machine-check a local contract-preservation result and the maximality of the selected candidate opening price.
- (C4) We isolate package-scale sharing as the core “no broken strategy” invariant and verify its algebraic core, while explicitly classifying the result as reduced because the executable complex matcher is not yet connected to it.
- (C5) We adapt Coelho’s statement-faithfulness discipline to this artifact through a declaration-level ledger, assumption printing, and kernel re-checking.

Scope. The first verified kernel should remain conservative. It should address market-mechanical correctness, not economic valuation. Static no-arbitrage restrictions such as call monotonicity and convexity across strikes are important, and option-pricing theory imposes shape restrictions on call prices as functions of strike [8]. However, those restrictions belong to a separate post-clearing price-system validator rather than to the core matching kernel. The present artifact is a partially mechanized baseline: only declarations marked *full* or *library wrapper* in Section 12 are claimed as completed results.

2 Related Work

2.1 Verified double-sided auctions

The closest formal baseline is the Coq-formalized double-auction literature. Natarajan, Sarswat, and Singh model bids, asks, transactions, quantities, timestamps, and prices; define matchings between duplicate-free bid and ask lists; define individual rationality as execution prices lying between bid and ask limits; define fairness in terms of competitiveness; and prove correctness of fair, uniform, and maximum matching mechanisms [1]. Their work is highly relevant because it already handles multi-quantity orders and extraction of verified OCaml/Haskell programs. Garg et al. continue this line with automated checkers and uniqueness results [2].

The present work differs in the object language and in the package-level invariants. The existing double-auction framework treats a generic product. In options, the product is a contract series. A direct match must preserve the full series identity. Furthermore, options venues support complex orders whose execution correctness is not reducible to independent single-leg matching.

2.2 Audited formalized mathematical finance

Coelho’s Lean 4 library develops a broad machine-checked foundation for mathematical finance, including stochastic calculus, derivative pricing, portfolio theory, risk measures, fixed income, actuarial mathematics, and decentralized finance [4]. Its subject matter is largely complementary to the matching problem studied here: Coelho verifies the mathematical and pricing layer, whereas this paper targets order-book and package-execution mechanics.

The methodological overlap is more important. Coelho classifies each result as *full*, *library wrapper*, *reduced core*, or *placeholder*, and couples the classification to an axiom audit. We adopt the same vocabulary in Section 12. This prevents a proof sketch, an admitted Coq declaration, or a theorem about a simplified algebraic core from being reported as an end-to-end verified matching mechanism. The audit is particularly valuable here because the prototype already contains meaningful checked lemmas alongside unfinished sorting, allocation, and complex-execution bridges.

2.3 Options-market microstructure

Listed options are organized by option class and series. OCC specifications describe standard equity option units, premium quotations, strike intervals, exercise style, exercise settlement, and expiration dates [5]. Exchange rulebooks then specify how orders and quotes are accepted, ranked, opened, matched, and protected. For example, Cboe’s opening process determines an opening trade price for a series when orders and quotes are marketable against each other; the stated VMIM criterion is volume-maximizing and imbalance-minimizing within the opening collar [6]. Cboe’s complex-order rules define concurrent multi-series execution, all-or-none instructions, legging behavior, and execution-price protections [6].

This rulebook structure motivates the formal properties introduced below. Contract consistency corresponds to series identity. Expiry validity corresponds to preventing trades after the contract is no longer tradable. Tick validity corresponds to exchange minimum-increment regimes. Leg consistency corresponds to the concurrent-execution nature of complex orders.

2.4 Static no-arbitrage and option price systems

The present paper does not attempt to verify a pricing model. Nevertheless, it is useful to distinguish matching correctness from no-arbitrage price-system correctness. In classical option theory, same-expiry calls should be decreasing and convex in strike under no-arbitrage restrictions; Ait-Sahalia and Duarte use such monotonicity and convexity restrictions in nonparametric option-pricing estimation [8]. These restrictions are cross-series properties of price surfaces. They are not identical to the local execution invariants of a matching engine. We therefore treat static no-arbitrage constraints as an optional layer in Section 11.

3 Preliminaries and Design Principles

We use discrete types throughout. This choice is both faithful to implementation and convenient for mechanization. Prices are integer ticks, times are discrete timestamps, and orders are finite records contained in finite lists. The model is intended for Coq formalization, so all predicates should be decidable whenever they are used computationally.

Let $\mathbb{P} \subseteq \mathbb{N}$ denote nonnegative integer prices measured in the smallest exchange-relevant unit. Let $\mathbb{T} \subseteq \mathbb{N}$ denote timestamps. Let $\mathcal{U}$ denote a finite or countable set of underlyings. Let `OrderId` and `TraderId` be decidable identifier types.

The development follows three design principles.

- (P1) **Series separation.** Simple matching is performed only within one option contract series. Cross-series interaction appears only in complex orders or optional no-arbitrage validation.
- (P2) **Integer arithmetic.** Prices, quantities, package limits, and tick predicates are discrete. No proof should depend on real-number rounding unless a separate rounding lemma is supplied.
- (P3) **Layered verification.** The matching, auction, and package layers expose checked invariants to their successors.

4 Formal Model of Option Contracts

Definition 4.1 (Option type and exercise style). *Let*

$$\text{OType} = \{\text{Call}, \text{Put}\}, \quad \text{Style} = \{\text{American}, \text{European}\}.$$

An element of OType records whether the option is a call or put. An element of Style records the exercise style. The exercise style is part of the contract identity even though the matching engine does not price early exercise.

Definition 4.2 (Option contract). *An option contract is a tuple*

$$C = (u, K, T, \tau, \sigma, \mu, \delta),$$

where $u \in \mathcal{U}$ is the underlying, $K \in \mathbb{P}$ is the strike, $T \in \mathbb{T}$ is the expiration timestamp, $\tau \in \text{OType}$, $\sigma \in \text{Style}$, $\mu \in \mathbb{N}$ is the multiplier, and $\delta : \mathbb{P} \rightarrow \text{Prop}$ is the tick-validity predicate.

Definition 4.3 (Contract equality). *Two contracts are equal when all contract fields agree:*

$$\begin{aligned} C_1 = C_2 \iff & u(C_1) = u(C_2) \wedge K(C_1) = K(C_2) \wedge T(C_1) = T(C_2) \\ & \wedge \tau(C_1) = \tau(C_2) \wedge \sigma(C_1) = \sigma(C_2) \wedge \mu(C_1) = \mu(C_2) \\ & \wedge \delta(C_1) = \delta(C_2). \end{aligned}$$

In Coq, the tick predicate should normally be represented by a decidable code or by a structure with proof-irrelevant equality assumptions, rather than by arbitrary functional equality.

Definition 4.4 (Tradable contract). *At time t , a contract C is tradable if*

$$\text{Tradable}(C, t) \iff \text{Listed}(C, t) \wedge t < T(C) \wedge \mu(C) > 0.$$

Here $\text{Listed}(C, t)$ abstracts exchange-specific listing rules.

Remark 4.5. *The tradability predicate deliberately separates expiration validity from listing validity. A contract can fail to be tradable because it has expired, because it is not listed, or because its multiplier is invalid. This separation is helpful for both proof debugging and executable error reporting.*

5 Simple Option Orders and Matchings

Definition 5.1 (Simple option order). *A simple option order is a tuple*

$$o = (s, C, q, p, t, \iota, \eta),$$

where $s \in \{\text{Buy}, \text{Sell}\}$ is the side, $C \in \mathcal{C}$ is the contract, $q \in \mathbb{N}$ is the submitted quantity, $p \in \mathbb{P}$ is the limit price, $t \in \mathbb{T}$ is the timestamp, $\iota \in \text{OrderId}$ is the order identifier, and η is an instruction such as day, IOC, FOK, or AON.

We write $\text{side}(o)$, $\text{ctr}(o)$, $\text{qty}(o)$, and $\text{lim}(o)$ for the corresponding fields.

Definition 5.2 (Transaction). *A simple option transaction is a tuple*

$$m = (b, a, q_m, p_m),$$

where b is a buy order, a is a sell order, $q_m \in \mathbb{N}$ is the executed quantity, and $p_m \in \mathbb{P}$ is the execution price.

Definition 5.3 (Filled quantity). *For an order o and transaction list M , define*

$$\text{filled}(o, M) = \sum_{m \in M: b(m)=o} q_m + \sum_{m \in M: a(m)=o} q_m.$$

Definition 5.4 (Valid simple matching). *Let B be a duplicate-free list of buy orders and A a duplicate-free list of sell orders. A transaction list M is a valid simple option matching for (B, A, t) if the following hold:*

- (M1) **Membership:** *every transaction uses a bid from B and an ask from A .*
- (M2) **Side correctness:** *each transaction pairs a buy order with a sell order.*
- (M3) **Positive quantity:** *every transaction has $q_m > 0$.*

(M4) **Contract consistency:** $\text{ctr}(b(m)) = \text{ctr}(a(m))$ for all $m \in M$.

(M5) **Tradability:** $\text{Tradable}(\text{ctr}(b(m)), t)$ for all $m \in M$.

(M6) **Limit compatibility:** $\lim(a(m)) \leq \lim(b(m))$ for all $m \in M$.

(M7) **No overfill:** $\text{filled}(o, M) \leq \text{qty}(o)$ for every $o \in B \cup A$.

(M8) **Tick validity:** $\delta_{\text{ctr}(b(m))}(p_m)$ for all $m \in M$.

Definition 5.5 (Individual rationality). *A matching M is individually rational if*

$$\forall m \in M, \quad \lim(a(m)) \leq p_m \leq \lim(b(m)).$$

Definition 5.6 (Contract consistency). *A matching M is contract-consistent if*

$$\forall m \in M, \quad \text{ctr}(b(m)) = \text{ctr}(a(m)).$$

Definition 5.7 (Expiry validity). *A matching M is expiry-valid at time t if*

$$\forall m \in M, \quad t < T(\text{ctr}(b(m))).$$

Definition 5.8 (Tick validity). *A matching M is tick-valid if*

$$\forall m \in M, \quad \delta_{\text{ctr}(b(m))}(p_m).$$

6 Price-Time Priority

For bids on the same contract, define competitiveness by

$$b_1 \succ_B b_2 \iff \lim(b_1) > \lim(b_2) \quad \text{or} \quad (\lim(b_1) = \lim(b_2) \wedge t(b_1) < t(b_2)).$$

For asks on the same contract,

$$a_1 \succ_A a_2 \iff \lim(a_1) < \lim(a_2) \quad \text{or} \quad (\lim(a_1) = \lim(a_2) \wedge t(a_1) < t(a_2)).$$

Definition 6.1 (Bid fairness). *A matching M is bid-fair if, for every two buy orders b_1, b_2 on the same contract,*

$$b_1 \succ_B b_2 \wedge \text{filled}(b_2, M) > 0 \implies \text{filled}(b_1, M) = \text{qty}(b_1).$$

Definition 6.2 (Ask fairness). *A matching M is ask-fair if, for every two sell orders a_1, a_2 on the same contract,*

$$a_1 \succ_A a_2 \wedge \text{filled}(a_2, M) > 0 \implies \text{filled}(a_1, M) = \text{qty}(a_1).$$

Definition 6.3 (Price-time fairness). *A matching is price-time fair if it is both bid-fair and ask-fair.*

Remark 6.4. *The specification chooses the price-time version. Some actual options venues use pro-rata, customer-priority overlays, or hybrid allocation rules in particular classes. A mature formalization should parameterize fairness by allocation policy. Price-time priority is chosen here because it is the cleanest starting point and the closest analogue of the formal double-auction literature.*

7 Single-Contract Continuous Matching: Specification and Proof Obligations

Fix a contract C . Define the filtered books

$$B_C = \{b \in B : \text{side}(b) = \text{Buy}, \text{ctr}(b) = C, \text{Tradable}(C, t)\},$$

$$A_C = \{a \in A : \text{side}(a) = \text{Sell}, \text{ctr}(a) = C, \text{Tradable}(C, t)\}.$$

The continuous matching procedure sorts B_C by descending bid competitiveness and A_C by descending ask competitiveness. It recursively matches the heads whenever

$$\lim(a) \leq \lim(b).$$

The executed quantity is

$$q_m = \min(q_{\text{rem}}(b), q_{\text{rem}}(a)),$$

and the price is chosen by a deterministic pricing rule

$$p_m = \text{PriceRule}(b, a),$$

subject to the precondition

$$\lim(a) \leq \text{PriceRule}(b, a) \leq \lim(b), \quad \delta_C(\text{PriceRule}(b, a)).$$

Algorithm MatchSeries(C, B, A, t).

1. Filter B and A to valid orders on contract C .
2. Sort bids by $\succ_B$ and asks by $\succ_A$.
3. While both lists are nonempty and the best ask is compatible with the best bid, trade the minimum residual quantity.
4. Remove fully filled orders and retain partially filled residual orders.
5. Return the generated transaction list.

Proof obligation 7.1 (Contract consistency of single-series matching). *For every contract C , book pair (B, A) , and time t , if*

$$M = \text{MatchSeries}(C, B, A, t),$$

then

$$\forall m \in M, \quad \text{ctr}(b(m)) = \text{ctr}(a(m)) = C.$$

Proof sketch. The algorithm emits transactions only from the filtered lists B_C and A_C . Every bid in B_C and every ask in A_C has contract C . The result follows by induction on the recursive construction of M . In the base case, no transaction is emitted. In the inductive case, the emitted head transaction is contract-consistent by the filter invariant, and the remaining transactions are contract-consistent by the induction hypothesis. $\square$

Proof obligation 7.2 (No overfill). *For every order $o \in B \cup A$,*

$$\text{filled}(o, \text{MatchSeries}(C, B, A, t)) \leq \text{qty}(o).$$

Proof sketch. Maintain the invariant

$$\text{qty}(o) = \text{filled}(o, M_{\text{prefix}}) + q_{\text{rem}}(o)$$

for every active order after every recursive step. A step trades the minimum of the residual quantities of the current best bid and ask, so no residual quantity becomes negative. Therefore $\text{filled}(o, M) \leq \text{qty}(o)$ for every order at termination. $\square$

Proof obligation 7.3 (Individual rationality and tick validity). *Assume the pricing rule satisfies*

$$\lim(a) \leq \text{PriceRule}(b, a) \leq \lim(b) \quad \text{and} \quad \delta_C(\text{PriceRule}(b, a))$$

whenever b and a are matched by MatchSeries. Then the output matching is individually rational and tick-valid.

Proof sketch. Every emitted transaction receives its price from PriceRule. The theorem follows by applying the pricing-rule precondition to each emitted transaction. $\square$

Proof obligation 7.4 (Price-time fairness). *The output MatchSeries(C, B, A, t) is price-time fair.*

Proof sketch. The algorithm consumes sorted bid and ask lists from most competitive to least competitive. A less competitive bid can be reached only after all more competitive bids have either been fully filled or no compatible ask remains. If the less competitive bid receives a positive fill, compatible liquidity existed at that step; hence all earlier bids must have been fully filled. The ask-side argument is symmetric. $\square$

8 Opening Auction for One Option Series: Specification and Proof Obligations

For a fixed contract C , an opening auction receives queued buy and sell orders before continuous trading begins. A uniform-price matching is a matching in which every transaction executes at the same price:

$$\forall m_1, m_2 \in M, \quad p_{m_1} = p_{m_2}.$$

For a candidate clearing price p , define executable demand and supply:

$$D_C(p) = \sum_{b \in B_C: \lim(b) \geq p} \text{qty}(b), \quad S_C(p) = \sum_{a \in A_C: \lim(a) \leq p} \text{qty}(a).$$

The executable volume is

$$V_C(p) = \min(D_C(p), S_C(p)),$$

and the imbalance is

$$I_C(p) = |D_C(p) - S_C(p)|.$$

Definition 8.1 (Opening price). *Let $\Pi_C \subseteq \mathbb{P}$ be the finite set of candidate tick-valid prices. The opening price p^* is selected lexicographically:*

(O1) *maximize $V_C(p)$;*

(O2) *among volume maximizers, minimize $I_C(p)$;*

(O3) *among remaining candidates, apply a deterministic tie-breaker.*

This abstraction mirrors the exchange-rule idea of a volume-maximizing, imbalance-minimizing opening price, while leaving exchange-specific collars and tie-breakers as parameters.

Definition 8.2 (Uniform valid matching). *A matching M is uniform-valid for (C, B, A, t) if it is a valid simple matching, every transaction is on contract C , every trade price equals a common price p , and every trade is individually rational at p .*

Proof obligation 8.3 (Maximum uniform clearing volume). *Let*

$$p^* = \text{OpenPrice}(C, B, A, t), \quad M^* = \text{OpenAuction}(C, B, A, t).$$

Then M^ is a uniform-valid matching at p^* , and for every uniform-valid matching M ,*

$$Q(M) \leq Q(M^*),$$

where $Q(M) = \sum_{m \in M} q_m$.

Proof sketch. Fix any uniform-valid matching M at price p . Every matched bid must have $\lim(b) \geq p$, and every matched ask must have $\lim(a) \leq p$. Hence

$$Q(M) \leq D_C(p), \quad Q(M) \leq S_C(p),$$

so

$$Q(M) \leq V_C(p).$$

Since p^* maximizes V_C , we have $V_C(p) \leq V_C(p^*)$. The construction of OpenAuction fills exactly $V_C(p^*)$ eligible contracts at price p^* , so $Q(M^*) = V_C(p^*)$. Therefore $Q(M) \leq Q(M^*)$. $\square$

Proof obligation 8.4 (Opening-auction correctness). *The output $\text{OpenAuction}(C, B, A, t)$ satisfies contract consistency, no overfill, individual rationality, tick validity, expiry validity, uniformity, and maximum uniform volume.*

Proof sketch. Contract consistency and expiry validity follow from the contract filter. No overfill follows from the same residual-quantity invariant used in continuous matching. Uniformity follows because every emitted transaction is assigned p^* . Individual rationality follows from eligibility at p^* . Tick validity follows from $p^* \in \Pi_C$. Maximum uniform volume is the previous theorem. $\square$

9 Complex Option Orders

A complex option order is a package of signed legs. The sign records whether the package buyer buys or sells the corresponding option contract. The absolute value records the leg ratio.

Definition 9.1 (Leg). *A leg is a pair*

$$\ell = (C, r),$$

where $C \in \mathcal{C}$ and $r \in \mathbb{Z} \setminus \{0\}$. Positive r denotes a buy leg and negative r denotes a sell leg from the perspective of the package buyer.

Definition 9.2 (Strategy). *A complex option strategy is a finite list*

$$\Theta = [(C_1, r_1), \dots, (C_n, r_n)]$$

with $n \geq 2$, $r_i \neq 0$, and each C_i tradable at the relevant time. Additional exchange-specific constraints may restrict repeated contracts, maximum leg count, allowed ratios, or underlying consistency.

Example 9.3 (Vertical call spread). *A one-by-one vertical call spread with common underlying and expiry is*

$$\Theta_{\text{vertical}} = [(C_{K_1,T}^{\text{Call}}, +1), (C_{K_2,T}^{\text{Call}}, -1)].$$

Example 9.4 (Long straddle). *A long straddle is*

$$\Theta_{\text{straddle}} = [(C_{K,T}^{\text{Call}}, +1), (C_{K,T}^{\text{Put}}, +1)].$$

Example 9.5 (Butterfly). *A call butterfly may be represented as*

$$\Theta_{\text{fly}} = [(C_{K_1,T}^{\text{Call}}, +1), (C_{K_2,T}^{\text{Call}}, -2), (C_{K_3,T}^{\text{Call}}, +1)].$$

Definition 9.6 (Complex order). *A complex order is a tuple*

$$O^\Theta = (\Theta, Q_\Theta, P_\Theta, t, \iota, \eta),$$

where Θ is a strategy, $Q_\Theta \in \mathbb{N}$ is the package quantity, $P_\Theta \in \mathbb{P}$ is the package limit price, t is the timestamp, ι is the package order identifier, and η is the instruction.

Definition 9.7 (Leg-consistent execution). *An execution X of O^Θ is leg-consistent if there exists a package scale $k \in \mathbb{N}$ such that*

$$\forall i \in \{1, \dots, n\}, \quad \text{legFilled}_i(X) = k|r_i|.$$

For an all-or-none order, $k \in \{0, Q_\Theta\}$. For a partial ratio-preserving order, $0 \leq k \leq Q_\Theta$.

Definition 9.8 (Net package price). *If leg i executes at price p_i , the signed net package price is*

$$\text{Net}(X) = \sum_{i=1}^n r_i p_i.$$

For a package buy order, net rationality requires $\text{Net}(X) \leq P_\Theta$. For a package sell order, it requires $\text{Net}(X) \geq P_\Theta$.

Remark 9.9. *The sign convention can be changed, but it must be fixed once and used consistently. In implementation, it is often convenient to represent package side separately from signed leg direction. The theorem statements are invariant under this refactoring.*

10 Complex-Order Matching Algorithm

The complex-order matching algorithm has three stages.

Stage 1: strategy validation. The predicate $\text{ValidStrategy}(\Theta, t)$ checks that the strategy has at least two legs, every leg ratio is nonzero, every leg contract is tradable at t , every leg uses a valid tick grid, and all exchange-specific leg-count and underlying restrictions hold.

Stage 2: executable-scale computation. For each leg i , let L_i be the available opposite-side liquidity for that leg, computed from the permitted liquidity sources: simple book, complex book, auction responses, or a venue-specific combination. The maximum feasible package scale is

$$k_{\max} = \min_i \left\lfloor \frac{L_i}{|r_i|} \right\rfloor.$$

Stage 3: ratio-preserving execution. If the instruction is AON, execute if and only if $k_{\max} \geq Q_{\Theta}$, then set $k = Q_{\Theta}$. If the instruction is FOK, perform the same test immediately and cancel otherwise. If partial ratio-preserving execution is allowed, set

$$k = \min(Q_{\Theta}, k_{\max}).$$

The algorithm emits leg transactions with quantities $k|r_i|$ for every leg i , or emits nothing.

Proof obligation 10.1 (Complex-order leg consistency). *Let*

$$X = \text{MatchComplex}(O^{\Theta}, \mathcal{B}, t)$$

be the execution generated by the complex-order matching algorithm against books $\mathcal{B}$. If X is nonempty, then there exists $k \in \mathbb{N}$ such that

$$\forall i, \quad \text{legFilled}_i(X) = k|r_i|.$$

If O^{Θ} is AON, then $k = Q_{\Theta}$. If partial ratio-preserving execution is permitted, then $0 < k \leq Q_{\Theta}$.

Proof sketch. The algorithm computes a single package scale k before emitting any component transaction. Every emitted leg quantity is obtained by multiplying $|r_i|$ by that same k . Therefore all component fills are normalized by the same package scale. In the AON branch, emission occurs only after checking $k_{\max} \geq Q_{\Theta}$, and the algorithm sets $k = Q_{\Theta}$. In the partial branch, $k = \min(Q_{\Theta}, k_{\max})$, so $k \leq Q_{\Theta}$. $\square$

Proof obligation 10.2 (No broken strategy). *For any complex order O^{Θ} , the algorithm cannot produce an execution X and legs i, j such that*

$$\text{legFilled}_i(X)|r_j| \neq \text{legFilled}_j(X)|r_i|$$

while both legs are required by Θ .

Proof sketch. By leg consistency, there exists a common package scale k such that $\text{legFilled}_i(X) = k|r_i|$ for all i . Both sides of the displayed inequality therefore equal $k|r_i||r_j|$. $\square$

Proof obligation 10.3 (Complex no-overfill). *For every complex order O^{Θ} ,*

$$\text{filledPackage}(O^{\Theta}, X) \leq Q_{\Theta},$$

and every leg satisfies

$$\text{legFilled}_i(X) \leq Q_{\Theta}|r_i|.$$

Proof sketch. Leg consistency gives $\text{legFilled}_i(X) = k|r_i|$. The algorithm always chooses $k \leq Q_{\Theta}$. Therefore $\text{legFilled}_i(X) \leq Q_{\Theta}|r_i|$. $\square$

Proof obligation 10.4 (Net package rationality). *Assume the complex pricing rule checks the package limit and component tick predicates before execution. Then every complex execution satisfies net package rationality and tick validity.*

Proof sketch. The algorithm emits a complex execution only after verifying the relevant net-price inequality and each component tick predicate. Since the emitted execution is exactly the checked execution, the result follows by unfolding the guard condition. $\square$

11 Optional Static No-Arbitrage Layer

The intended matching layer addresses mechanical correctness. Even when completed, it will not guarantee that the entire cross-section of option prices is arbitrage-free. A separate validator may check static restrictions such as monotonicity and convexity across strikes.

For calls with the same underlying and expiry,

$$K_1 < K_2 \implies C(K_1, T) \geq C(K_2, T).$$

For puts,

$$K_1 < K_2 \implies P(K_1, T) \leq P(K_2, T).$$

Discrete convexity of calls can be stated as

$$\frac{C(K_2, T) - C(K_1, T)}{K_2 - K_1} \leq \frac{C(K_3, T) - C(K_2, T)}{K_3 - K_2}, \quad K_1 < K_2 < K_3.$$

Equivalently, for equally spaced strikes,

$$C(K_1, T) - 2C(K_2, T) + C(K_3, T) \geq 0.$$

Remark 11.1. *These constraints should not be mixed into the first matching correctness theorem. A matching engine can mechanically execute orders correctly even when the displayed option chain contains an arbitrage opportunity. Conversely, a no-arbitrage repair procedure can adjust or flag cross-series price surfaces without changing the correctness of individual trade execution.*

12 Rocq Prototype, Faithfulness Audit, and Completion Plan

The accompanying prototype is a single Rocq source file plus an audit module and a declaration-level CSV ledger. It builds under Rocq 9.1.1. The ledger contains 17 proof declarations: 10 are classified *full* or *library wrapper*, one is a *reduced core*, and six are *placeholders* implemented with **Admitted**. Helper lemmas are included in these counts; the paper-facing status is summarized in Table 1.

The audit module runs **Print Assumptions** on the completed headline results. Those results are closed under the global context. A separate **rocq check** invocation rechecks the compiled module and reports the six admitted declarations as axioms in the module context. This distinction matters: kernel checking establishes the completed local results, while the ledger prevents incomplete declarations elsewhere in the file from being silently counted as verified contributions.

The reproducible commands are:

```
make build
make audit
```

The remainder of this section records the intended modular interfaces. These snippets are completion targets, not claims about the current prototype.

12.1 Module Contract

```
Inductive option_type := Call | Put.
Inductive exercise_style := American | European.
```

Claim or component	Status	Exact checked scope
Recursive-step contract preservation	Full	<code>match_step_contract_consistency</code> preserves a common contract assuming both residual books already contain only that contract.
End-to-end series contract consistency	Placeholder	The bridge through filtering and sorting is admitted.
End-to-end no overfill	Placeholder	The residual-quantity invariant and the final <code>match_series</code> theorem are admitted.
Limit respect and price-time fairness	Placeholder	The limit-respect lemma is admitted; fairness is not yet declared.
Candidate opening-price maximality	Full	<code>clearing_price_maximal</code> maximizes the executable-volume function over an explicit nonempty candidate list.
Opening-auction mechanism	Library wrapper / placeholder	<code>opening_auction_optimal</code> restates candidate-price maximality; no allocation-producing opening-auction function is yet verified.
No-broken-strategy invariant	Reduced core	<code>no_broken_strategy</code> is checked for fills defined directly from one shared package scale; it is not yet connected to an executable complex matcher.
Per-leg liquidity bound	Placeholder	<code>executed_fill_no_overfill</code> is admitted.

Table 1: Faithfulness audit of the paper-facing claims.

```
Record contract := {
  underlying : UnderlyingId;
  strike : price;
  expiry : time;
  otype : option_type;
  style : exercise_style;
  multiplier : nat;
  tick_valid : price -> Prop
}.
```

```
Definition expiry_valid (now : time) (c : contract) : Prop :=
  now < expiry c.
```

```
Definition tradable (now : time) (c : contract) : Prop :=
  listed now c /\ expiry_valid now c /\ multiplier c > 0.
```

A practical implementation should avoid arbitrary functional equality on `tick_valid`. One option is to store a finite tick-regime code and separately interpret it as a predicate.

12.2 Module Order

```
Inductive side := Buy | Sell.
Inductive instruction := Day | IOC | FOK | AON | PartialPackage.

Record order := {
  oside : side;
```

```

octr : contract;
oqty : nat;
olimit : price;
ots : time;
oid : order_id;
instr : instruction
}.

```

12.3 Module SimpleMatching

```

Record trade := {
  tbid : order;
  task : order;
  tqty : nat;
  tprice : price
}.

Definition contract_consistent (M : list trade) : Prop :=
  forall m, In m M -> octr (tbid m) = octr (task m).

Definition individual_rational (M : list trade) : Prop :=
  forall m, In m M ->
    olimit (task m) <= tprice m /\ tprice m <= olimit (tbid m).

```

Representative theorem targets:

```

Theorem match_series_contract_consistent :
  forall C B A now,
    contract_consistent (match_series C B A now).

Theorem match_series_no_overfill :
  forall C B A now o,
    filled o (match_series C B A now) <= oqty o.

Theorem match_series_IR :
  forall C B A now,
    individual_rational (match_series C B A now).

Theorem match_series_fair :
  forall C B A now,
    price_time_fair (match_series C B A now).

```

12.4 Module OpeningAuction

```

Definition demand_at (C : contract) (B : list order) (p : price) : nat :=
  sum_qty (filter (fun b => same_contract (octr b) C && (p <=? olimit b)) B).

Definition supply_at (C : contract) (A : list order) (p : price) : nat :=
  sum_qty (filter (fun a => same_contract (octr a) C && (olimit a <=? p)) A).

Definition executable_volume C B A p :=
  Nat.min (demand_at C B p) (supply_at C A p).

```

Main theorem target:

```
Theorem opening_auction_max_uniform :  
  forall C B A now M,  
    uniform_valid C now M ->  
    total_qty M <= total_qty (open_auction C B A now).
```

12.5 Module ComplexOrder

```
Record leg := {  
  lctr : contract;  
  ratio : Z  
}.  
  
Record complex_order := {  
  legs : list leg;  
  pkg_qty : nat;  
  pkg_limit : price;  
  pkg_ts : time;  
  pkg_id : order_id;  
  pkg_instr : instruction  
}.
```

Core theorem target:

```
Theorem complex_leg_consistency :  
  forall O books now X,  
    X = match_complex O books now ->  
    executed X ->  
    exists k,  
      forall l,  
        In l (legs O) ->  
        filled_leg X l = k * Z.to_nat (Z.abs (ratio l)).
```

12.6 Module Certificates

The final module should define compact certificates for exchange or regulatory checking. A simple matching certificate may contain per-order fill quantities and trade prices. A complex certificate may contain the package id, package scale k , and component leg fills. The checker theorem should state:

If the verified checker accepts a certificate, then the corresponding execution satisfies the required matching invariants.

13 Current Evaluation and Completion Tests

The current evaluation is proof-oriented. A clean build elaborates the prototype and audit module; `Print Assumptions` confirms that the four completed headline declarations named in the audit module do not depend on admitted declarations. The kernel checker accepts the compiled development and separately exposes the six placeholders in the transitive module context. This

is deliberately weaker than an end-to-end mechanism evaluation: there is not yet an extracted matcher, an adversarial-book generator, or a reference-engine comparison.

Once the placeholders in Table 1 are discharged, the executable implementation should be evaluated in three additional ways.

Synthetic adversarial books. Generate books with expired contracts, invalid ticks, duplicate order identifiers, zero quantities, incompatible contracts, crossed books, partially fillable orders, and adversarial timestamp patterns. The verified checker should reject malformed outputs and accept outputs produced by the certified mechanism.

Strategy-template tests. Generate vertical spreads, calendars, diagonals, straddles, strangles, butterflies, collars, and covered calls. For each strategy, deliberately generate broken executions such as filling one leg but not the other, or filling legs in incorrect ratios. The complex checker should reject all broken executions.

Reference-engine comparison. Compare an unverified reference implementation against the verified program extracted from Coq. The comparison should follow the strategy of verified double-auction checkers: if the reference output disagrees with the verified output on per-order fill quantities under the same formal rule, the reference implementation violates the specified rule or implements a different rule.

14 Discussion

The proposed framework isolates the part of options trading that is best suited to formal verification: matching-engine correctness. A matching engine should satisfy hard invariants regardless of volatility, trader beliefs, pricing model, or hedging strategy. An order should not be overfilled. A buyer should not pay above the bid limit. A seller should not receive below the ask limit. A trade should not print on the wrong option series. An expired contract should not trade. A complex order should not become a broken strategy.

The cleanest target architecture is layered:

single-series matching $\longrightarrow$ opening auction $\longrightarrow$ complex-order package execution.

In the completed development, single-series matching should establish contract identity, no overfill, individual rationality, tick validity, and fairness. Opening-auction verification should add uniform price, a concrete allocation, and maximum executable volume. Complex-order matching should then add ratio-preserving package execution. The present prototype checks only the cores identified in Table 1; the arrows above are a dependency plan, not a completed theorem graph.

15 Limitations and Future Work

Allocation policies. This draft uses price-time fairness. Real options venues may use pro-rata, customer-priority overlays, or hybrid allocation models. Future work should define an abstract allocation-policy interface and prove correctness for multiple policy instances.

Venue-specific complex-order rules. Real rulebooks contain many details: complex order auctions, legging restrictions, stock-option orders, priority customers, auctions, collars, drill-through protections, and class-specific minimum increments. This paper captures the core package invariant but does not attempt to reproduce every venue-specific rule.

Mechanization status. The prototype is buildable, but it is not complete: six declarations are admitted, no program is extracted, and several paper-level properties have no declaration yet. The ledger is therefore part of the artifact rather than an editorial appendix. A submission claiming an end-to-end verified mechanism should wait until the placeholder count is zero and the headline theorems are audited against the intended statements.

No-arbitrage validation. Static no-arbitrage validation is intentionally outside the first kernel. A natural second paper would verify monotonicity, convexity, calendar-spread consistency, put-call parity, and box-spread constraints over finite option chains.

16 Conclusion

This paper presents a verification blueprint and an audited Rocq prototype for option-market matching. The model extends ordinary double-auction objects with option-series identity and signed multi-leg strategies. Its central design invariant is package-scale sharing: all required legs of a complex execution must be filled in the prescribed ratios.

The current artifact establishes useful but limited cores—notably recursive-step contract preservation, maximal selection from a finite candidate-price set, and the algebraic ratio-preservation core. End-to-end no overfill, limit respect, opening allocation, fairness, and the executable complex-order bridge remain explicit proof obligations. Reporting that boundary is a substantive improvement borrowed from the faithfulness discipline of the Lean mathematical-finance reference library.

The central message survives the narrower claim: option-market verification should begin with microstructure correctness, not pricing. Once the matching kernel is complete, static no-arbitrage, volatility-surface consistency, margin rules, and portfolio-risk checks can be added as separately audited layers.

References

- [1] Raja Natarajan, Suneel Sarawat, and Abhishek Kr. Singh. Verified double sided auctions for financial markets. *arXiv preprint arXiv:2104.08437*, 2021. Available at <https://arxiv.org/abs/2104.08437>.
- [2] Mohit Garg, N. Raja, Suneel Sarawat, and Abhishek Kr. Singh. Double auctions: Formalization and automated checkers. *arXiv preprint arXiv:2410.18751*, 2024. Available at <https://arxiv.org/abs/2410.18751>.
- [3] Suneel Sarawat and Abhishek Kr. Singh. Formally verified trades in financial markets. *arXiv preprint arXiv:2007.10805*, 2020. Available at <https://arxiv.org/abs/2007.10805>.
- [4] Raphael Coelho. A formally verified library of mathematical finance in Lean 4. *arXiv preprint arXiv:2606.01356*, 2026. Available at <https://arxiv.org/abs/2606.01356>.

- [5] The Options Clearing Corporation. Equity options product specifications. Available at <https://www.theocc.com/clearance-and-settlement/clearing/equity-options-product-specifications>.
- [6] Cboe Exchange, Inc. Cboe Options Exchange Rule Book. Available at https://cdn.cboe.com/resources/regulation/rule_book/C1_Exchange_Rule_Book.pdf.
- [7] U.S. Securities and Exchange Commission, Division of Trading and Markets. Roundtable on options market structure. Available at <https://www.sec.gov/files/roundtable-options-market-structure.pdf>.
- [8] Yacine Aït-Sahalia and Jefferson Duarte. Nonparametric option pricing under shape restrictions. *NBER Working Paper No. 8944*, 2002. Available at <https://www.nber.org/papers/w8944>.
- [9] Douglas T. Breeden and Robert H. Litzenberger. Prices of state-contingent claims implicit in option prices. *The Journal of Business*, 51(4):621–651, 1978.